CASCADE

Change Analysis for Software Cascade using Dependency Evidence

> *Understand every consequence before every merge.*

AI-Powered
Blast Radius Analyzer

Evidence-backed
Product Knowledge Graph for Explainable Pull Request Impact Analysis

Presented by
Harsh Vardhan
B.Tech, Computer Science & Engineering (Data Science & Analytics)

Indian Institute of Information Technology Nagpur

AI Engineer Assignment
Testsigma • July 2026

THE PROBLEM

Every Pull Request Has Hidden Consequences

Modern software systems consist of thousands of interconnected components. A seemingly small code change can silently propagate across the product, affecting user journeys, APIs, documentation, and business requirements.

Hidden Dependencies

Relationships between requirements, UI, backend services, and source code are rarely documented end-to-end.

Limited Visibility

Traditional code review focuses on changed files—not the product-wide impact of those changes.

Production Risk

Developers often discover broken user flows, outdated documentation, or failing **integrations only after** deployment.

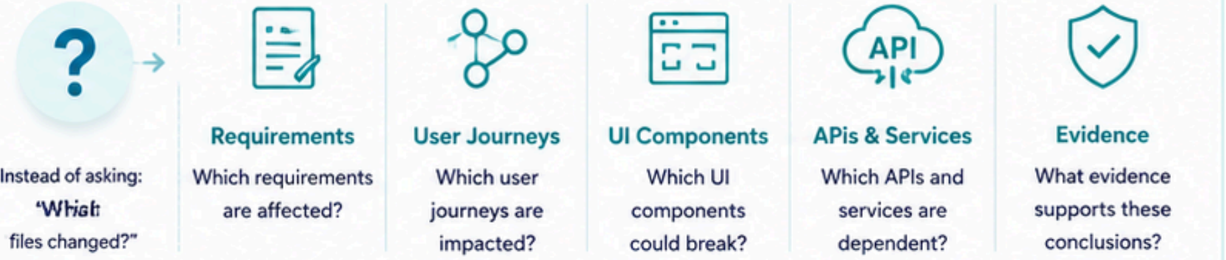
Growing Complexity

As products evolve, understanding the full consequence of a single Pull Request becomes increasingly difficult.

EXISTING WORKFLOW



WHAT TEAMS ACTUALLY NEED



> ***Understanding code changes is easy.***
> ***Understanding their product impact is hard.***

3

The Solution

Introducing CASCADE

An AI-powered system that constructs an **Evidence-backed Product Knowledge Graph** by combining multiple sources of truth.

INPUTS

Product Documentation
- README
- PRDs
- Wikis

Live Application
- Pages
- DOM
- User Flows

Source Repository
- Components
- APIs
- Database

Pull Request
Code changes, diffs and metadata

CASCADE

Evidence-backed Product Knowledge Graph



AI + Graph Traversal + Reasoning
to discover semantic dependencies and infer impact

OUTPUT

An explainable **Blast Radius Report** showing

- ✓ **Impacted Requirements** — Which requirements and features are affected
- ✓ **Affected UI** — Pages, components, and user flows impacted
- ✓ **Backend Services** — APIs, services, controllers, databases affected
- ✓ **Confidence Scores** — AI-generated confidence for each impact
- ✓ **Supporting Evidence** — Graph paths, code references, and documentation links

Core Idea | **Understand the product—not just the code.**

4 System Architecture

From Raw Data to Explainable Impact Analysis

CASCADE orchestrates multiple data sources to build a **knowledge graph** and reason about the **impact** of every Pull Request.

INPUT SOURCES

Documentation
- README
- PRDs
- Wikis
- Feature Specs

Live Application
- Pages
- DOM Structure
- Screenshots
- User Flows

Source Code
- Components
- APIs
- Database Models
- Configurations

Pull Request
- Git Diff
- Changed Files
- Commit Info
- PR Metadata

1 Data Extraction & Parsing

- **Documentation Parser** — LLMs + NLP to extract requirements & entities
- **Autonomous Web Crawler** — Browser agent (Playwright) to capture UI artifacts
- **Static Code Analyzer** — AST parsing, symbol extraction, call graph

2 Evidence-backed Product Knowledge Graph



Requirements ↔ UI ↔ Components ↔ APIs ↔ Database

All entities and relationships stored in Neo4j

3 Pull Request Analysis Engine

- **Git Diff Processing** — Identify changed files and code blocks
- **AST & Symbol Extraction** — Functions, classes, APIs, imports
- **Dependency Mapping** — Map changes to graph nodes

4 Graph Traversal + AI Reasoning

- **Dependency Traversal** — Follow edges to find impacted entities
- **Semantic Linking** — LLM validates and enriches relationships
- **Confidence Scoring** — Assign confidence based on evidence & signals

5 Explainable Blast Radius Report

- Affected Features
- User Journeys
- UI Components
- Backend Services
- Confidence Scores
- Supporting Evidence

Human-readable report with graph paths, code references, and evidence snippets

CORE COMPONENTS

Knowledge Extraction
Converts unstructured data from docs, UI and code into structured entities.

Product Knowledge Graph
Unified graph connecting requirements, UI, code, APIs and data.

AI Reasoning Engine
Traverses the graph to predict downstream impact using AI + graph algorithms.

Explainable Reporting
Generates an evidence-backed report with impact, confidence and reasoning.

One Unified Graph. | Multiple Data Sources. | Explainable Impact Analysis **Before Every Merge.**

5

Building the Product Knowledge Graph

The Single Source of Product Intelligence

CASCADE transforms disconnected engineering artifacts into a unified, **evidence-backed** Product Knowledge Graph.

KNOWLEDGE SOURCES

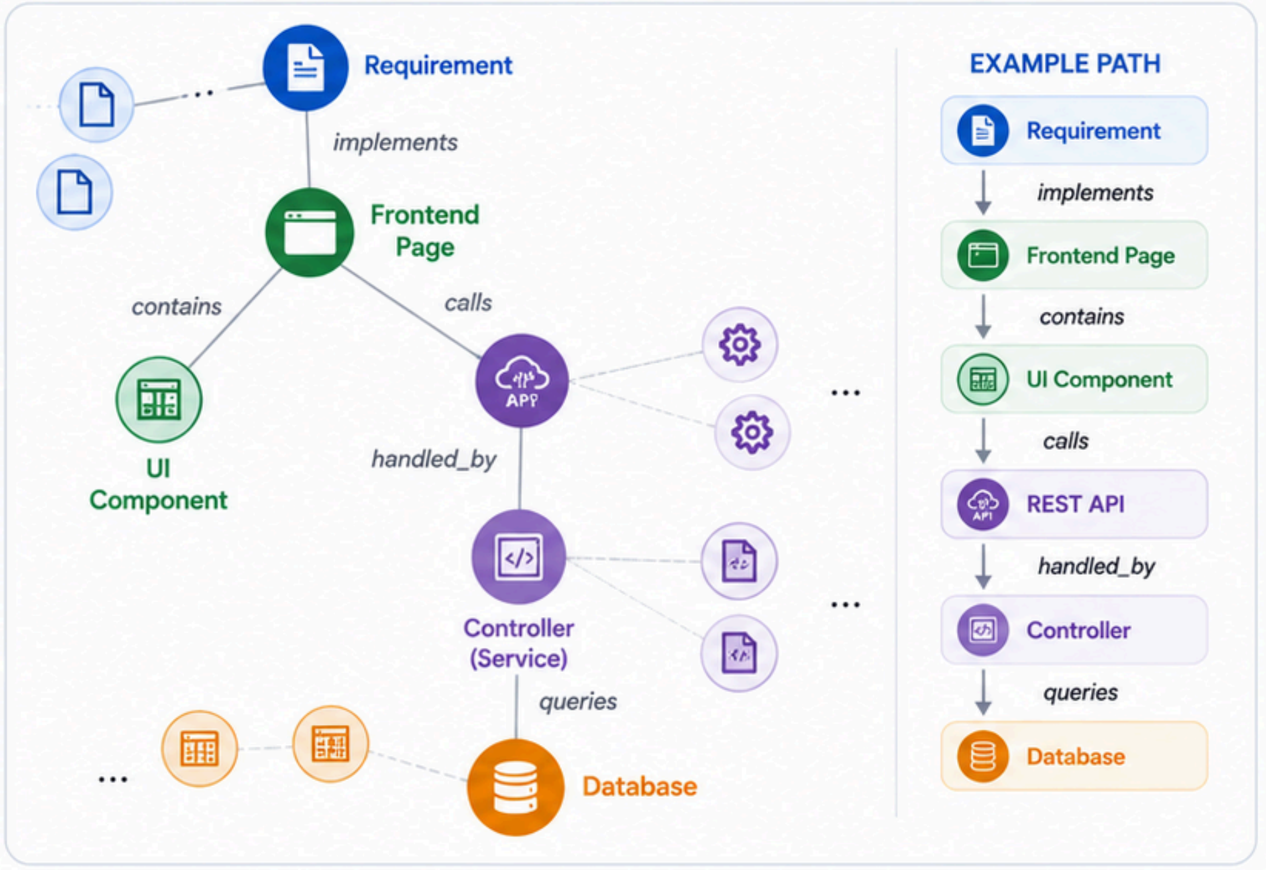
Product Layer
- Product Requirements (PRDs)
- README & Documentation
- Feature Specifications
- User Stories

Experience Layer
- Pages & Screens
- User Flows
- DOM Elements
- UI Components

Implementation Layer
- Source Code
- APIs
- Services
- Database Models

GRAPH CONSTRUCTION

Every artifact becomes a **node**, while relationships become **typed edges**.



EXAMPLE PATH

WHY A KNOWLEDGE GRAPH?

Multi-hop Traversal
Discover indirect dependencies across multiple layers of the stack.

Cross-layer Traceability
Trace connections from business requirements to database entities.

Semantic Linking
Link semantically related entities across documentation, UI and code.

Explainable Reasoning
Every relationship is explicit, enabling transparent and trustworthy impact analysis.

A Pull Request changes code. | A Knowledge Graph reveals what that code means to the entire product.

6

Pull Request Analysis

Understanding the Change Before Understanding the Impact

Every Pull Request becomes the **starting point** for product-wide impact analysis.

INPUT: PULL REQUEST

Example PR
Refactor authentication middleware

#3821 Open

Git Diff
+243 -128

Changed Files
8 files changed

Modified Functions
12 functions modified

Commit Metadata
Author, message, timestamp, labels

Added / Removed Symbols
Classes, methods, variables, imports

PR ANALYSIS PIPELINE

1 Analyze the Pull Request

Ingest and parse the PR to understand the raw changes.

```
@@ -45,7 +45,9 @@
- if (!user) {
-   throw new Error();
- }
+ if (!user || !user.isActive) {
+   throw new AuthError();
+ }
  return next();
```

- Syntax-aware diff
- File classification
- Change summarization
- Metadata enrichment

2 Identify Changed Entities

Static code analysis maps the changes to meaningful software entities.

- React Components
- Backend Services
- REST APIs
- Database Models
- Configuration Files

Result: Set of affected entities extracted from the change.

3 Locate in the Knowledge Graph

Match the entities with their corresponding nodes in the Product Knowledge Graph.



Entity resolution using
- ✓ Exact match
- ✓ Semantic similarity
- ✓ Alias & naming mapping

4 Start Dependency Traversal

The changed nodes become the entry points for graph traversal.



- Parent Dependencies
- Child Dependencies
- Cross-layer Relationships
- Semantic Links

GRAPH TRAVERSAL STARTS

Traversal begins from changed nodes to discover all downstream impacts.

Components
Affected UI & code components

APIs
Impacted endpoints and contracts

Services
Backend services and modules

Database
Tables, schemas, queries

Requirements
Linked product requirements

User Flows
Affected user journeys and interactions

Traditional Code Review Asks:
"What changed?"

CASCADE Asks:
"What does this change affect across the entire product?"

Every Pull Request becomes a graph query that reveals hidden product dependencies before code is merged.

7 Graph Traversal & AI Reasoning

Discovering the True Blast Radius

Starting from the changed entities, CASCADE traverses the Product Knowledge Graph to uncover downstream product impact.

1 Changed Nodes (Entry Points)

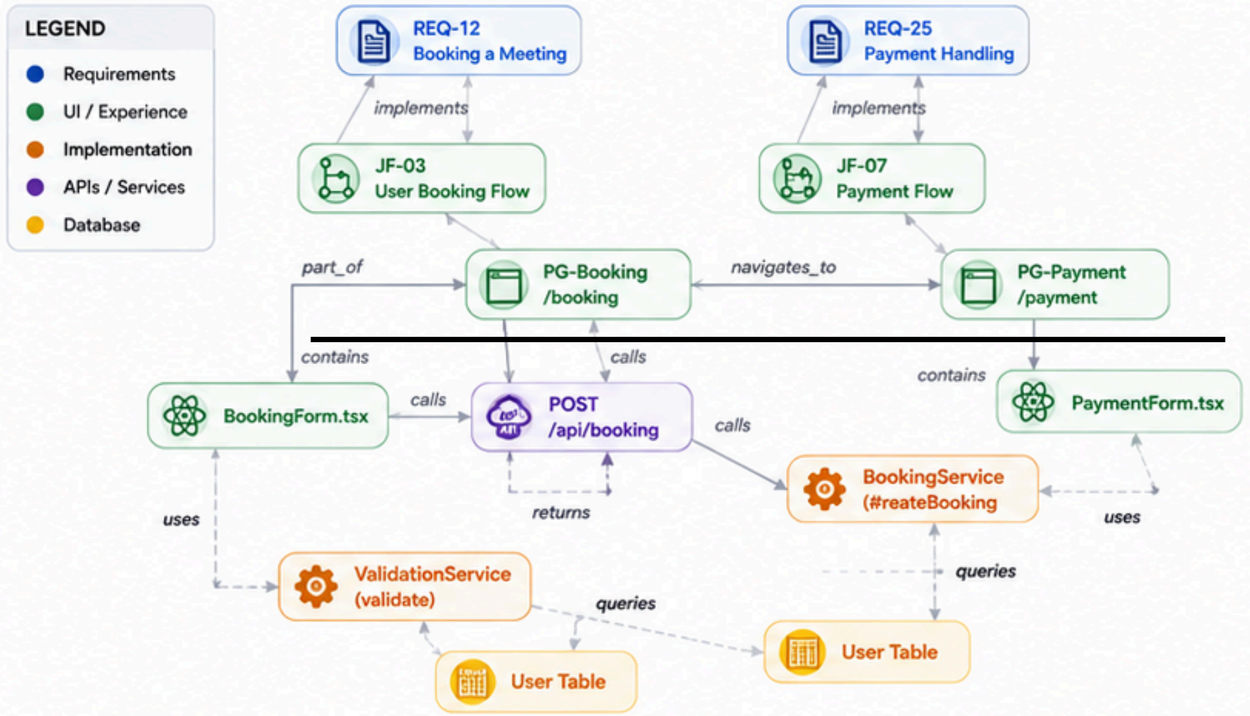
Entities extracted from the Pull Request become seed nodes in the knowledge graph.

- **React Component** — BookingForm.tsx — Changed
- **REST API** — POST /api/booking — Changed
- **Backend Service** — BookingService — Changed
- **Database Model** — Booking — Changed

Change Summary
8 files changed
12 functions modified
3 new functions added

2 Multi-hop Graph Traversal

CASCADE follows relationships across multiple layers to identify affected components.



Why This Matters

Traditional dependency analysis only follows direct code references.

A → B → C

CASCADE reasons across documentation, user experience, and implementation to uncover indirect, semantic impacts.

A → B → C → D → E

3 AI-powered Semantic Reasoning

AI enriches and validates relationships beyond explicit code references.

- **Semantic Relationship Validation** — Validate if the relationship truly exists in the product context.
- **Cross-layer Entity Linking** — Link semantically equivalent entities across layers.
- **Documentation Context Retrieval** — Fetch relevant docs, READMEs, PRDs, and wikis for context.
- **Confidence Score Estimation** — Assign confidence based on evidence and graph signals.

4 Evidence-backed Impact Prediction

CASCADE predicts and ranks all impacted entities.

Affected Features / Requirements	92%
Affected User Journeys	88%
Affected UI Components / Pages	87%
Affected Backend Services / APIs	93%
Affected Database Tables / Models	90%

Every impact is supported by graph paths and evidence snippets.

Graph algorithms discover connections. | AI explains why they matter.

8

AI-Generated Blast Radius Report

From Graph Intelligence to Actionable QA Decisions

CASCADE transforms graph analysis into an explainable, evidence-backed report that helps QA teams focus on the highest-risk areas before deployment.

EXECUTIVE SUMMARY

Example: PR #3046

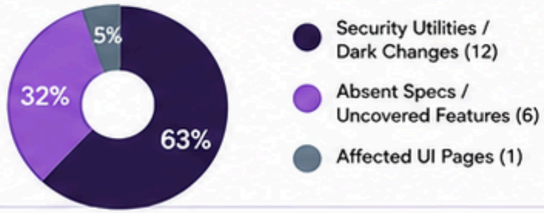
PR Type:
Security & Permission Hardening

Target PR:
documenso/documenso #3046
fix: tighten permission and validation checks

Analysis Summary

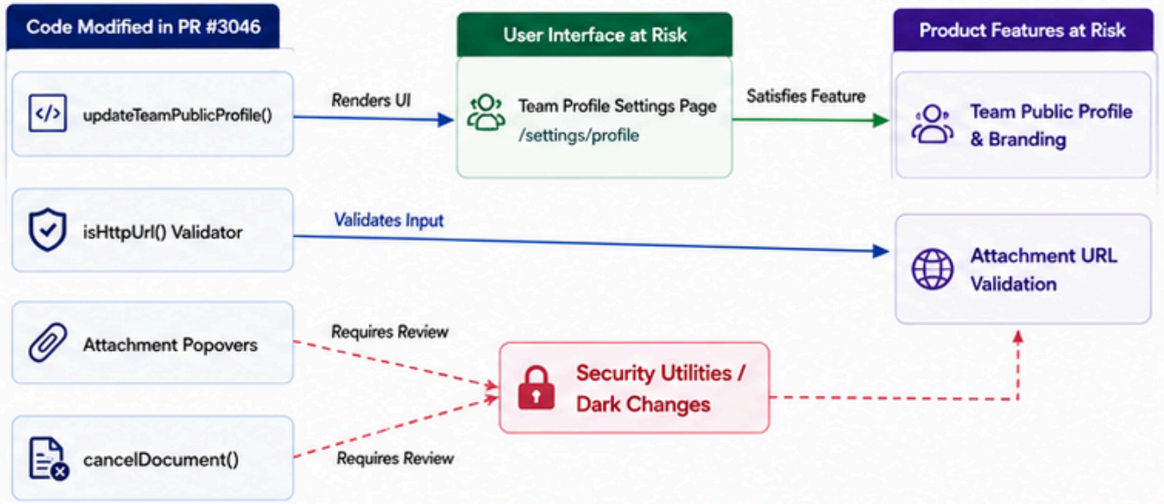
Files Changed	29
Affected UI Page	1
Security Utility (Dark) Changes	12
Uncovered Specifications	6

Risk Overview Breakdown for PR #3046



IMPACT TRAVERSAL: HOW CODE CHANGES REACH THE UI

CASCADE maps how modified code connects to user-facing pages and product features.



Impact paths are derived from the Product Knowledge Graph and validated using static + semantic analysis.

AI-GENERATED QA TEST PLAN

CASCADE generates prioritized manual test scenarios with step-by-step guidance.

1. **Team Public Profile**
   Verify profile updates and admin permissions.
2. **Attachment URL Validation**
   Test HTTP vs HTTPS link validation.
3. **Attachment Upload / Edit / Delete**
   Verify upload, edit and delete permissions.
4. **Document Cancellation & Deletion**
   Validate owner vs non-owner permissions.
5. **Zapier Webhooks**
   Test webhook subscription and unsubscription.
6. **Password Reset Flow**
   Verify password reset email and reset functionality.

6 priority test cases to ensure release quality.

SUMMARY METRICS

29	Files Changed
1	UI Pages Affected
6	Priority QA Tests
12	Security Utility (Dark) Changes
6	Uncovered Specifications

Why This Matters

- Focus QA effort on the highest-risk areas
- Prevent regressions in critical security flows
- Ship with confidence and complete visibility

Evidence-backed analysis. Prioritized testing. Confident releases.

9

Live Demonstration

End-to-End Workflow in Action

Let's see how CASCADE analyzes a real Pull Request and generates an explainable Blast Radius Report.

Repository
documenso/documenso

Target Pull Request
#3046
Security & Permission Hardening

LIVE PIPELINE: CASCADE WORKFLOW

1 Repository Analysis

- Clone repository
- Parse project structure
- Build code relationships
- Identify modules and dependencies

2 Documentation Understanding

- Process README and docs
- Extract product entities
- Identify feature specifications
- Build requirement map

3 Application Discovery

- Crawl live application
- Capture pages and routes
- Extract UI components
- Map user flows and journeys

4 Knowledge Graph Construction

- Merge docs, UI & code
- Create graph nodes (Features, UI, APIs, Services, DB, Req.)
- Create relationships
- Store in Neo4j

5 Pull Request Analysis

- Ingest PR #3046
- Parse Git diff
- Identify changed files
- Extract changed functions & entities

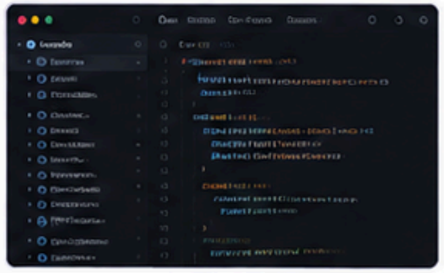
6 Graph Traversal & AI Reasoning

- Multi-hop graph traversal
- Cross-layer dependency discovery
- Semantic reasoning with LLM
- Confidence scoring

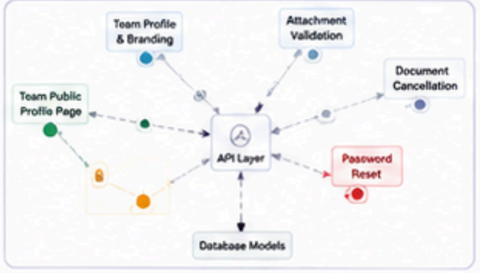
7 Blast Radius Report

- Executive Summary
- Impact Traversal
- Priority QA Tests
- Summary Metrics
- Evidence & References

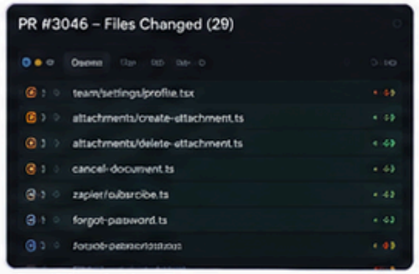
WHAT YOU'LL SEE IN THE DEMO



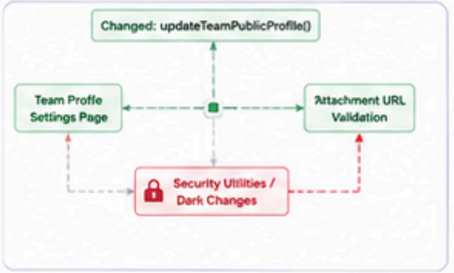
Repository Indexing
Parsing codebase and building structural relationships



Product Knowledge Graph
Unified view of docs, UI, code and data dependencies



Pull Request Analysis
Git diff parsing and changed entity extraction



AI Reasoning Process
Graph traversal, dependency discovery & impact prediction



Final Blast Radius Report
Explainable insights, test plan and summary metrics

One Pull Request. One Knowledge Graph. Complete Product Impact Visibility.

10

Technical Highlights & Future Roadmap

Engineering Decisions Behind CASCADE

Designed to provide **explainable, scalable, and extensible** blast radius analysis for modern software systems.

KEY INNOVATIONS

Evidence-backed Product Knowledge Graph

Unifies documentation, UI, source code, and Pull Requests into a single graph for product-level reasoning.

AI + Graph Intelligence

Combines graph traversal with semantic reasoning to identify indirect dependencies that traditional static analysis cannot detect.

Explainable Predictions

Every impacted feature is backed by:

- Graph traversal paths
- Code references
- Documentation evidence
- Confidence scores

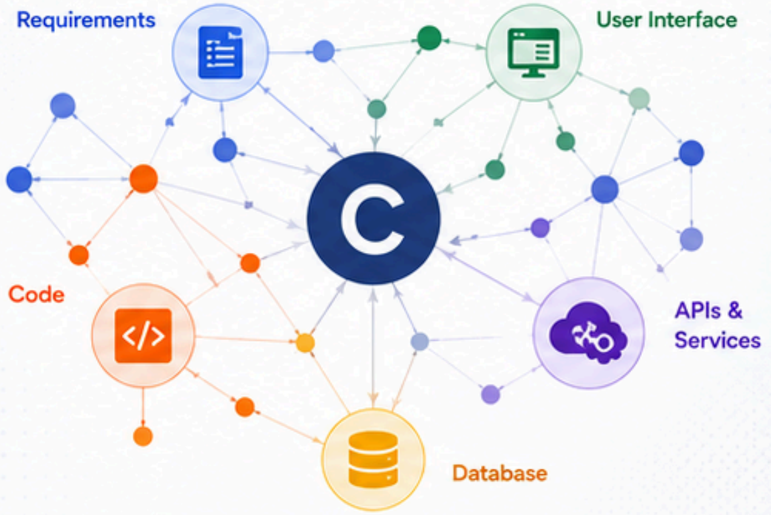
QA-first Output

Automatically generates:

- Prioritized regression test scenarios
- Impact summaries
- Evidence-backed recommendations
- Actionable reports for QA engineers

CASCADE

Change Analysis for Software Cascade using Dependency Evidence



One Knowledge Graph.
Complete Product Impact Visibility.

CURRENT CAPABILITIES

- Repository Analysis
- Documentation Understanding
- Application Crawling
- Knowledge Graph Construction
- Pull Request Analysis
- Graph-based Reasoning
- Blast Radius Report Generation

FUTURE ROADMAP

CI/CD Integration
Automatically analyze every Pull Request before merge.

Continuous Knowledge Graph Updates
Keep the graph synchronized with evolving repositories and documentation.

Multi-Repository Support
Analyze dependencies across microservices and distributed systems.

Intelligent Regression Test Selection
Recommend the minimum set of tests required based on predicted impact.

Enterprise Scale & Customization
Role-based insights, custom rules, and advanced governance for large engineering organizations.

CASCADE transforms Pull Request review from code-level inspection into product-level impact intelligence.

Explainable & Trustworthy | Scalable & Efficient | Extensible & Future-ready

THANK YOU!

CASCADE

Change Analysis for Software Cascade using Dependency Evidence

Understand Impact
Map code changes to product-wide impact

Explain with Evidence
Every prediction backed by graph paths & proof

Prioritize Testing
Focus on what matters most for QA

Ship with Confidence
Make safer releases with complete visibility

> Every Pull Request should answer one question before it is merged:
>
> **“What will this change impact?”**

QUESTIONS?